

PROGRAMMATION FONCTIONNELLE 2 : TD9 PRÉDICATS INDUCTIFS ET RÉFLEXION BOOLÉENNE avril 2026 — Pierre Rousselin

Exercice 1 : Décider l'égalité sur les entiers naturels de Peano

1. Définir la fonction `eqb : nat -> nat -> bool` qui retourne `true` si et seulement si ses deux arguments sont égaux.
2. Prouver (Rocq) le lemme suivant :
`Lemma eqb_refl (n : nat) : (n =? n) = true.`
3. En déduire :
`Lemma eq_eqb (n m : nat) : n = m -> (n =? m) = true.`
4. Prouver (papier ou Rocq) :
`Lemma eqb_eq (n m : nat) : (n =? m) = true -> n = m.`
5. Définir la fonction `mem : nat -> list nat -> bool` qui vaut `true` si et seulement si son premier argument apparaît dans son second.
6. Prouver (papier ou Rocq) le lemme suivant :
`Lemma neq_hd_mem (x h : nat) (t : list nat) :
x <> h -> mem x (h :: t) = true -> mem x t = true.`

--- * ---

Correction de l'exercice 1 :

1. `Fixpoint eqb (n m : nat) :=
 match n, m with
 | 0, 0 => true
 | 0, _ => false
 | _, 0 => false
 | S p, S q => eqb p q
 end.
Notation "n =? m" := (eqb n m).`
2. `Lemma eqb_refl (n : nat) : (n =? n) = true.
Proof.
 induction n as [| p IH].
 - reflexivity.
 - simpl. exact IH.
Qed.`
3. `Lemma eq_eqb (n m : nat) : n = m -> (n =? m) = true.
Proof.
 intros H; rewrite H; exact (eqb_refl _).
Qed.
(* Ou alors, si on a très très bien compris eq *)
Lemma eq_eqb' (n m : nat) : n = m -> (n =? m) = true.
Proof.
 intros H. destruct H. exact (eqb_refl _).
Qed.`
4. `Lemma eqb_eq (n m : nat) : (n =? m) = true -> n = m.
Proof.
 induction n as [| p IH] in m |- *.
 - destruct m as [| m'].
 + intros _; reflexivity.
 + intros H; discriminate H.
 - destruct m as [| m'].
 + intros H; discriminate H.`

```

    + simpl. intros H. specialize (IH _ H).
      rewrite IH. reflexivity.
Qed.
5. Fixpoint mem (n : nat) (l : list nat) :=
  match l with
  | [] => false
  | h :: t => eqb n h || mem n t
  end.
6. Lemma neq_hd_mem (x h : nat) (t : list nat) :
  x <> h -> mem x (h :: t) = true -> mem x t = true.
Proof.
  intros H.
  simpl.
  destruct (eqb x h) eqn:E.
  - apply eqb_eq in E.
    exfalso; apply H; exact E.
  - simpl; intros H'; exact H'.
Qed.

```

--- * ---

Exercice 2 : Décider l'égalité des listes d'entiers naturels

1. Définir la fonction `eqb : list nat -> list nat -> bool` qui retourne `true` si et seulement si ses deux arguments sont égaux. Pour l'égalité booléenne sur `nat`, prendre celle de l'exercice précédent en l'appelant `Nat.eqb`.
2. Faire de même que dans l'exercice précédent pour établir la réflexion entre `eqb` et `eq` sur les listes d'entiers naturels? On peut utiliser le lemme suivant :
`Lemma andb_prop : forall a b : bool, a && b = true -> a = true /\ b = true`
3. De quoi avons-nous besoin pour définir une égalité booléennes entre listes et pour établir la réflexion entre cette égalité et l'égalité usuelle?

--- * ---

Correction de l'exercice 2 :

```

1. Fixpoint eqb (l1 l2 : list nat) :=
  match l1, l2 with
  | [], [] => true
  | _, [] | [], _ => false
  | h1 :: t1, h2 :: t2 => Nat.eqb h1 h2 && eqb t1 t2
  end.
2. Lemma eqb_refl (l : list nat) : eqb l l = true.
Proof.
  induction l as [| h t IH].
  - reflexivity.
  - simpl. rewrite Nat.eqb_refl IH. reflexivity.
Qed.

Lemma eq_eqb (l1 l2 : list nat) : l1 = l2 -> eqb l1 l2 = true.
Proof.
  intros ->. exact (eqb_refl l2).
Qed.

Lemma eqb_eq (l1 l2 : list nat) : eqb l1 l2 = true -> l1 = l2.
Proof.
  induction l1 as [| h t IH] in l2 |- *.

```

```
- destruct 12; [reflexivity | discriminate].
- destruct 12 as [| h2 t2].
+ discriminate.
+ simpl.
  intros H. apply andb_prop in H as [H1 H2].
  apply Nat.eqb_eq in H1. rewrite H1.
  apply IH in H2. rewrite H2.
  reflexivity.
```

Qed.

3. On n'a pas utilisé les propriétés de `nat` à part l'existence de `eqb` et le fait que `eqb` se réfléchisse dans `eq` sur `nat`. Tout type dans lequel on a une égalité décidable ferait donc l'affaire.

--- * ---